

GUÍA PRÁCTICA 2

ARQUITECTURA POR CAPAS CON FASTAPI, SQLALCHEMY Y POSTGRESQL

Programación Web / Desarrollo Full Stack
Nivel intermedio • Individual / Parejas • 2–3 horas

Proyecto: API de gestión de usuarios con persistencia

1. Presentación

En la Guía 1 se construyó una API con FastAPI utilizando una lista de usuarios en memoria. En esta segunda etapa se dará un paso hacia una solución más cercana a un proyecto profesional: los datos se almacenarán en PostgreSQL y la aplicación se organizará mediante una arquitectura por capas.

Se utilizará SQLAlchemy como ORM para comunicar FastAPI con PostgreSQL. Además, se separarán las responsabilidades en archivos diferentes para que el código sea más fácil de entender, mantener y ampliar.

2. Objetivo general

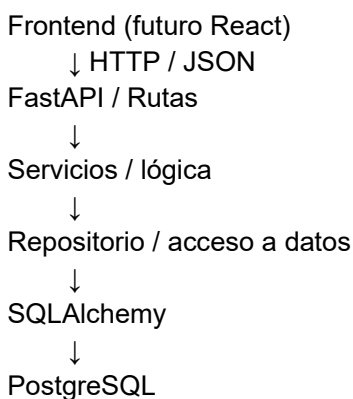
Implementar una API REST de gestión de usuarios utilizando FastAPI, SQLAlchemy y PostgreSQL, aplicando una arquitectura básica por capas para separar configuración de base de datos, modelos, esquemas, lógica de acceso a datos y rutas.

3. Objetivos específicos

- Crear una base de datos PostgreSQL para el proyecto.
- Configurar una conexión segura y reutilizable mediante SQLAlchemy.
- Crear el modelo Usuario que represente la tabla usuarios.
- Crear automáticamente la tabla mediante SQLAlchemy.
- Separar las responsabilidades del proyecto en archivos y capas.
- Implementar operaciones para listar, consultar, registrar y eliminar usuarios.
- Utilizar Pydantic para validar los datos de entrada y salida.
- Probar la API mediante Swagger/OpenAPI.
- Comprender la diferencia entre almacenar datos en memoria y almacenarlos permanentemente.

4. Arquitectura propuesta

La estructura será sencilla para fines didácticos. No se busca implementar una Clean Architecture completa, sino enseñar el principio fundamental de separación de responsabilidades.



5. Estructura de carpetas

```
proyecto_usuarios/  
├──  
├── main.py  
├── database.py  
├── models.py  
├── schemas.py  
├── crud.py  
├── requirements.txt  
└── .env
```

Para facilitar el aprendizaje, se mantiene una estructura pequeña. En proyectos más grandes, cada capa puede convertirse en una carpeta con varios módulos.

6. Responsabilidad de cada archivo

Archivo	Responsabilidad	No debería encargarse de
main.py	Crear FastAPI, configurar CORS y definir endpoints.	Conexiones SQL complejas o detalles internos de PostgreSQL.
database.py	Conexión, motor, sesiones y Base de SQLAlchemy.	Reglas de usuarios o rutas HTTP.
models.py	Definir la estructura de la tabla Usuario.	Recibir peticiones HTTP.
schemas.py	Validar datos de entrada y salida con Pydantic.	Guardar directamente en PostgreSQL.
crud.py	Realizar operaciones de usuarios mediante SQLAlchemy.	Crear rutas FastAPI.
.env	Guardar configuración sensible, como contraseña.	Código de la aplicación.

7. Paso 1 — Crear la base de datos PostgreSQL

Abra pgAdmin o psql. Cree una base de datos llamada universidad. La contraseña Admin1234 se utiliza en esta guía únicamente como ejemplo local; no debe publicarse en GitHub.

SQL:

```
CREATE DATABASE universidad;
```

La tabla usuarios NO se creará manualmente. SQLAlchemy la generará a partir del modelo.

8. Paso 2 — Crear el entorno virtual

En la carpeta del proyecto:

```
python -m venv venv
```

```
PowerShell: .\venv\Scripts\Activate.ps1
```

```
CMD: venv\Scripts\activate
```

9. Paso 3 — Instalar dependencias

Crear requirements.txt:

```
fastapi
uvicorn[standard]
sqlalchemy
psycopg[binary]
pydantic[email]
python-dotenv
```

Instalar:

```
pip install -r requirements.txt
```

10. Paso 4 — Configurar .env

Crear un archivo llamado .env:

```
DATABASE_URL=postgresql+psycopg://postgres:Admin1234@localhost:5432/universidad
```

No publique este archivo en repositorios. Para un proyecto real se recomienda utilizar variables de entorno y secretos administrados por el entorno de despliegue.

11. Paso 5 — Crear database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from dotenv import load_dotenv
import os
```

```
load_dotenv()
```

```
DATABASE_URL = os.getenv("DATABASE_URL")
```

```
engine = create_engine(DATABASE_URL)
```

```
SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False
)
```

```
Base = declarative_base()
```

```
def get_db():
    db = SessionLocal()

    try:
        yield db
    finally:
        db.close()
```

get_db() abre una sesión para una petición y garantiza que se cierre al terminar.

12. Paso 6 — Crear models.py

```
from sqlalchemy import Column, Integer, String
from database import Base
```

```
class Usuario(Base):

    __tablename__ = "usuarios"

    id = Column(Integer, primary_key=True, index=True)

    nombre = Column(String(100), nullable=False)

    apellido = Column(String(100), nullable=False)

    telefono = Column(String(20), nullable=False)

    edad = Column(Integer, nullable=False)
```

Este modelo representa la tabla usuarios. Cada Column representa una columna de PostgreSQL. El id es la clave primaria.

13. Paso 7 — Crear schemas.py

```
from pydantic import BaseModel
```

```
class UsuarioCreate(BaseModel):
    nombre: str
    apellido: str
    telefono: str
    edad: int

class UsuarioResponse(BaseModel):
    id: int
    nombre: str
    apellido: str
    telefono: str
    edad: int

class Config:
    from_attributes = True
```

Los schemas controlan qué datos recibe la API y qué datos devuelve. Esto evita mezclar directamente el modelo de base de datos con los datos que llegan desde el cliente.

14. Paso 8 — Crear crud.py

```
from sqlalchemy.orm import Session
from models import Usuario
from schemas import UsuarioCreate
```

```
def obtener_usuarios(db: Session):
```

```

        return db.query(Usuario).all()

def obtener_usuario(db: Session, usuario_id: int):
    return (
        db.query(Usuario)
        .filter(Usuario.id == usuario_id)
        .first()
    )

def crear_usuario(db: Session, datos: UsuarioCreate):

    nuevo_usuario = Usuario(
        nombre=datos.nombre,
        apellido=datos.apellido,
        telefono=datos.telefono,
        edad=datos.edad
    )

    db.add(nuevo_usuario)
    db.commit()
    db.refresh(nuevo_usuario)

    return nuevo_usuario

def eliminar_usuario(db: Session, usuario_id: int):

    usuario = obtener_usuario(db, usuario_id)

    if usuario is None:
        return None

    db.delete(usuario)
    db.commit()

    return usuario

```

Este archivo concentra las operaciones de acceso a datos. Las rutas de FastAPI no necesitan conocer los detalles de `db.query()`, `db.add()`, `db.delete()`, etc.

15. Paso 9 — Crear main.py

```

from fastapi import FastAPI, Depends, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session

from database import Base, engine, get_db
from schemas import UsuarioCreate, UsuarioResponse
import crud

# Crear las tablas si no existen
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="API Usuarios Universitaria",

```

```

        version="1.0.0"
    )

# CORS para el futuro Frontend React
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",
        "http://127.0.0.1:5173"
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"]
)

@app.get("/")
def inicio():
    return "Bienvenido al programa usuarios"

@app.get(
    "/listadeusuarios",
    response_model=list[UsuarioResponse]
)
def obtener_usuarios(db: Session = Depends(get_db)):

    return crud.obtener_usuarios(db)

@app.post(
    "/agregarusuarios",
    response_model=UsuarioResponse,
    status_code=201
)
def agregar_usuario(
    datos: UsuarioCreate,
    db: Session = Depends(get_db)
):

    return crud.crear_usuario(db, datos)

@app.get(
    "/listadeusuarios/{id}",
    response_model=UsuarioResponse
)
def obtener_usuario(
    id: int,
    db: Session = Depends(get_db)
):

    usuario = crud.obtener_usuario(db, id)

    if usuario is None:
        raise HTTPException(
            status_code=404,
            detail="Usuario no encontrado"
        )

    return usuario

```

```

@app.delete("/eliminarusuario/{id}")
def eliminar_usuario(
    id: int,
    db: Session = Depends(get_db)
):

    usuario = crud.eliminar_usuario(db, id)

    if usuario is None:
        raise HTTPException(
            status_code=404,
            detail="Usuario no encontrado"
        )

    return {
        "mensaje": "Usuario eliminado exitosamente"
    }

```

16. ¿Qué ocurre cuando inicia la aplicación?

Al ejecutar Uvicorn, main.py importa los modelos y ejecuta Base.metadata.create_all(bind=engine). SQLAlchemy revisa la base de datos y crea la tabla usuarios si todavía no existe.

Flujo:

main.py → database.py → engine → PostgreSQL

main.py → models.py → Base.metadata.create_all() → tabla usuarios

17. Paso 10 — Ejecutar la API

Desde la carpeta del proyecto:

uvicorn main:app --reload

Abrir:

<http://127.0.0.1:8000/docs>

18. Probar POST /agregarusuarios

En Swagger, seleccionar POST /agregarusuarios, pulsar Try it out y enviar un JSON como:

```

{
  "nombre": "Carlos",
  "apellido": "Ramírez",
  "telefono": "3001234567",
  "edad": 30
}

```

La diferencia con la Guía 1 es fundamental: ahora el usuario no se agrega a una lista. Se convierte en un objeto SQLAlchemy y se guarda permanentemente en PostgreSQL mediante db.commit().

19. Comprobar la tabla en PostgreSQL

En pgAdmin, actualizar Schemas → public → Tables. Debe aparecer usuarios.

También puede ejecutar:

```
SELECT * FROM usuarios;
```

20. Pruebas obligatorias

- ☐ GET / → verificar mensaje.
- ☐ GET /listadeusuarios → inicialmente puede estar vacía.
- ☐ POST /agregarusuarios → registrar al menos tres usuarios.
- ☐ GET /listadeusuarios → comprobar los tres registros.
- ☐ GET /listadeusuarios/1 → consultar un usuario existente.
- ☐ GET /listadeusuarios/999 → debe responder HTTP 404.
- ☐ DELETE /eliminarusuario/2 → eliminar un usuario.
- ☐ GET /listadeusuarios → comprobar que el registro fue eliminado.
- ☐ Detener Uvicorn, iniciarlo nuevamente y comprobar que los usuarios siguen en PostgreSQL.

21. Ejercicios para el aprendiz

- Agregar una ruta GET /cantidadusuarios que consulte el total desde PostgreSQL.
- Crear una función CRUD para actualizar un usuario mediante PUT.
- Validar que la edad no pueda ser negativa.
- Agregar el campo ciudad al modelo, schema y CRUD.
- Investigar qué diferencia existe entre db.add(), db.commit() y db.refresh().
- Explicar por qué database.py no debe contener rutas FastAPI.
- Explicar por qué models.py no debe recibir directamente una petición HTTP.
- Conectar posteriormente el Frontend React de la actividad.

22. Buenas prácticas de seguridad

- No publicar la contraseña de PostgreSQL en GitHub.
- Usar .env para configuración local y agregar .env al .gitignore.
- No permitir cualquier origen CORS en producción sin una razón.

- No guardar contraseñas de usuarios en texto plano cuando se implemente autenticación.
- Validar los datos recibidos antes de guardarlos.
- No confiar en que el Frontend hará todas las validaciones.

23. Problemas frecuentes

Error de conexión a PostgreSQL

Compruebe que PostgreSQL esté iniciado, que la base universidad exista y que usuario, contraseña, puerto y nombre de BD sean correctos.

ModuleNotFoundError

Compruebe que está en la carpeta correcta y que instaló requirements.txt.

No aparece la tabla

Revise que models.py se importe desde main.py y que Base.metadata.create_all(bind=engine) se ejecute.

Contraseña incorrecta

Revise DATABASE_URL en .env.

Los datos no aparecen en Swagger

Compruebe que POST haya respondido 201 y luego ejecute GET.

24. Evidencias de aprendizaje

- Estructura de carpetas del proyecto.
- Archivo .env configurado localmente (sin mostrar la contraseña en la evidencia).
- Captura de la base de datos universidad.
- Captura de la tabla usuarios creada.
- Captura de Swagger con los endpoints.
- Captura de un POST exitoso.
- Captura de GET mostrando registros.
- Captura de DELETE exitoso.
- Consulta SELECT * FROM usuarios; mostrando los datos.
- Breve explicación de la función de cada capa.

25. Criterios de evaluación sugeridos

Criterio	Valor	Descripción
PostgreSQL y conexión	25%	Base creada y conexión SQLAlchemy funcionando.
Modelo y tabla	20%	Usuario correctamente definido y tabla creada.
Arquitectura por capas	20%	Responsabilidades separadas

		en archivos.
Endpoints	20%	GET, POST y DELETE funcionando.
Pruebas y evidencias	10%	Swagger y PostgreSQL comprobados.
Buenas prácticas	5%	Código ordenado y configuración protegida.

26. Reflexión final

Responda con sus propias palabras: ¿qué ventaja tiene separar `main.py`, `database.py`, `models.py`, `schemas.py` y `crud.py`? ¿Qué diferencia existe entre la lista de la Guía 1 y PostgreSQL? ¿Qué función cumple SQLAlchemy? ¿Por qué no debemos colocar toda la lógica del sistema dentro de `main.py`?

27. Resultado esperado

Al finalizar, el aprendiz tendrá una API que mantiene la idea sencilla de la primera guía, pero ahora utiliza persistencia real y separación de responsabilidades:

React (futuro) → FastAPI → CRUD → SQLAlchemy → PostgreSQL → usuarios